# Behavior of the New Krotov Variants in the Two-Moons QML Optimization Benchmark

Codex experiment report

March 30, 2026

## Experimental setup

The report analyzes the Krotov-family optimizers in the existing two-moons QML benchmark without changing the model or dataset pipeline. The setup uses 4 qubits, 3 trainable layers, 5 random seeds, and 100 outer iterations on the same train/test split convention as the original benchmark. The main comparison uses:

- `krotov_online`: the preserved single-sample stale-adjoint update with constant step size 0.3,
- `krotov_batch`: the new full-batch Krotov-inspired update with constant step size 0.3.

The fair accounting axis is the propagation cost

$$\text{cost units} = \text{sample forward passes} + \text{sample backward passes},$$

which treats one single-sample forward propagation and one single-sample adjoint propagation as the primitive work units. Mean-loss evaluations and gradient-direction evaluations are logged separately in the raw JSON files, but the report uses cost units when interpreting optimization speed.

## Main empirical findings

1. The online Krotov baseline still shows the aggressive initial descent that motivated the original implementation. Averaged over seeds, its loss after the first optimization step is 0.382, compared with 0.702 for the full-batch variant.
2. The full-batch update is markedly smoother but substantially less aggressive at the baseline step size. Its mean final loss is 0.436 compared with 0.371 for the online variant.
3. The online method pays for its fast initial progress with persistent late-stage oscillation. Its mean tail loss standard deviation over the last 20 logged points is 0.028, versus 0.004 for the batch variant.
4. The batch update is strongly step-size limited rather than obviously broken. Across the sweep, the best mean final loss comes from `batch, lr=1.0`, which reaches 0.365. This is materially better than the baseline batch setting and indicates that the default batch step size was too conservative for this problem.

## Quantitative comparison of the main variants

| Variant | Final loss | Final test acc. | Loss @ step 1 | Loss @ step 10 | Tail std. | Mean tail upda |
|---|---|---|---|---|---|---|
| online, lr=0.3 | 0.371 ± 0.034 | 0.856 ± 0.040 | 0.382 | 0.345 | 0.028 | 1.510 |
| batch, lr=0.3 | 0.436 ± 0.048 | 0.867 ± 0.018 | 0.702 | 0.629 | 0.004 | 0.014 |

Table 1: Mean behavior of the main Krotov variants over the configured seeds. Tail statistics are computed from the last 20 logged optimization points.

The table makes the central contrast explicit. The online variant is a high-gain optimizer: it descends quickly, reaches the lower mean final loss in the baseline comparison, and keeps relatively large updates even late in the run. The batch variant is much more regularized by construction because it averages the gate-wise contributions over the whole dataset before applying an update. That removes sample-order noise and the stale-adjoint inconsistency across successive samples, but it also suppresses the large coordinated moves that made the online rule effective at the beginning of training.

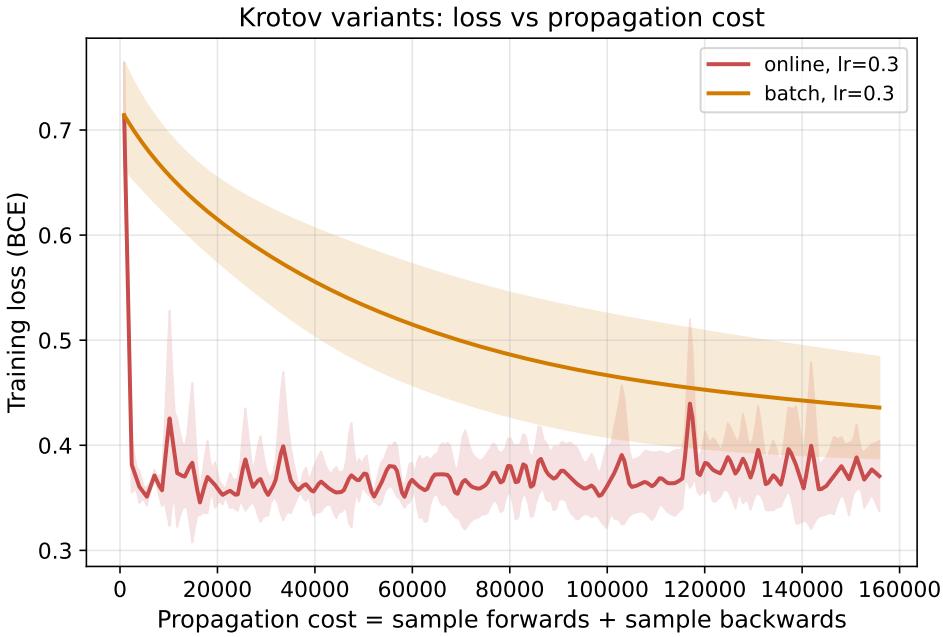


Figure 1: Training loss versus the fair propagation-cost metric for the main Krotov variants.

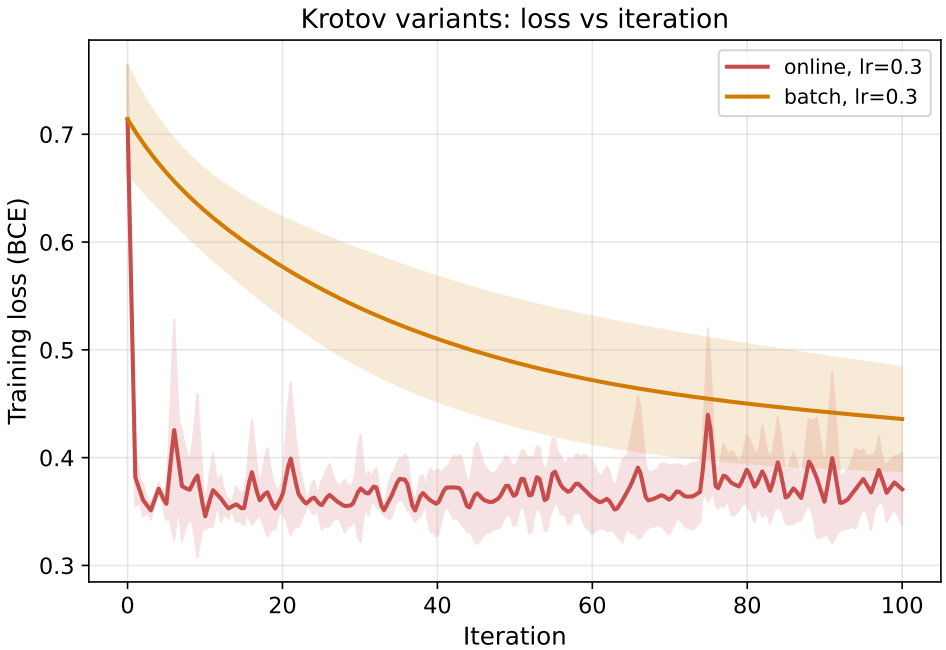


Figure 2: Training loss versus iteration for the main Krotov variants.

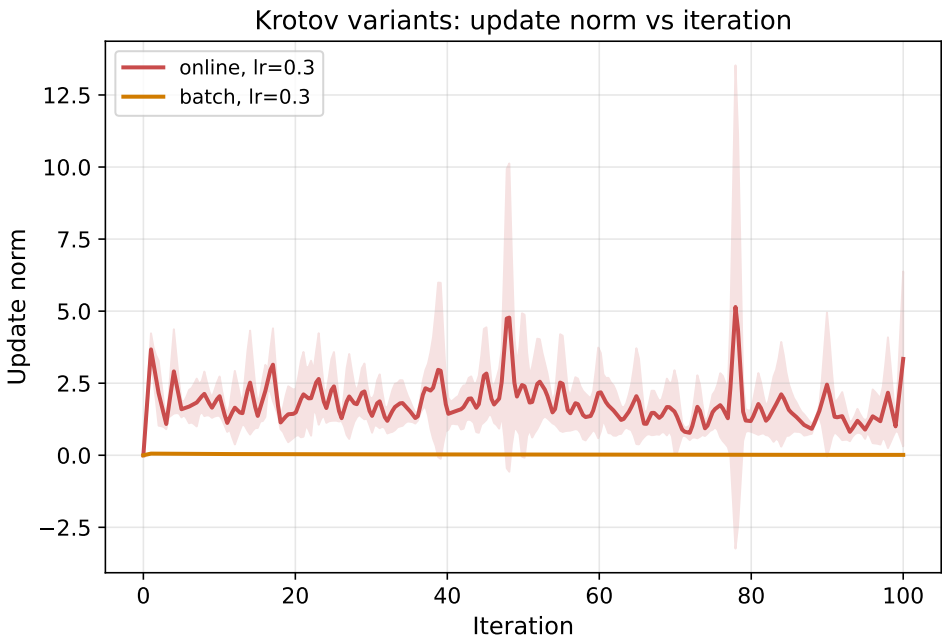


Figure 3: Update norms reveal that the online rule keeps moving aggressively while the batch rule contracts to much smaller parameter changes.

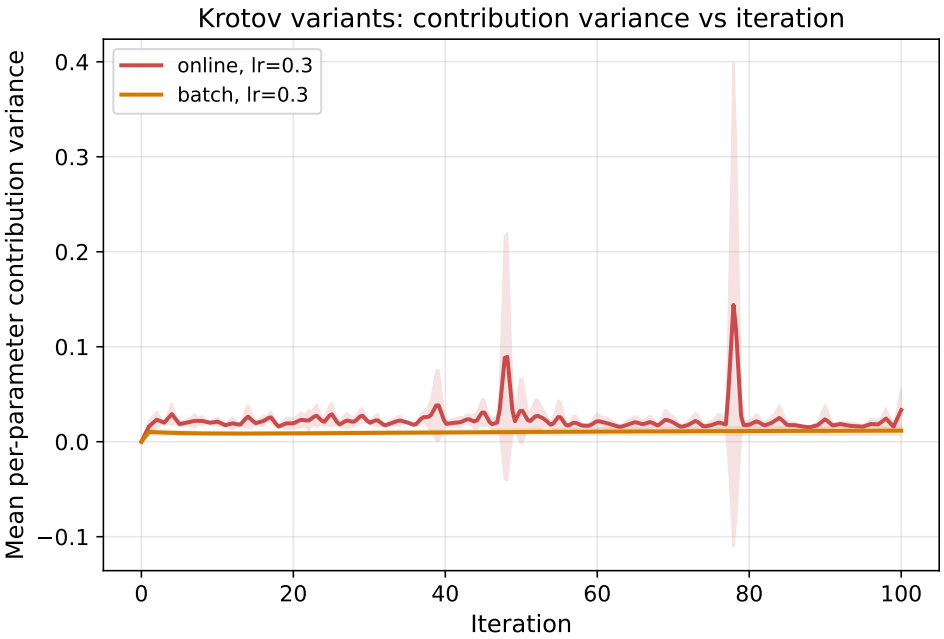


Figure 4: Per-parameter contribution variance. The online rule sees much noisier per-sample directions, which helps explain both the fast initial drop and the later oscillation.

## Interpretation of the optimization dynamics

The new diagnostics support a fairly specific interpretation of the Krotov behavior.

First, the online optimizer's rapid initial improvement is real under fair accounting. The loss decreases sharply in both the loss-versus-cost and loss-versus-iteration views. This means that the earlier observation was not an artifact of the old function-evaluation counter. The online rule is genuinely capable of finding a useful direction very quickly on this task.

Second, the same mechanism that makes the online method aggressive also makes it unstable. Each update is built from one sample at a time, and the backward co-states are computed before the subsequent gate-wise parameter changes. As a result, the method keeps taking relatively large, sample-specific steps even after it reaches a reasonably good decision boundary. The elevated contribution variance and tail update norms are consistent with exactly this picture.

Third, the full-batch variant behaves more like deterministic gradient descent in the gate-local coordinates. It aligns the update with the true benchmark objective, because the gate-wise contributions are averaged over the full training loss before a single parameter update is applied. This removes stale-adjoint accumulation across samples and collapses the sample-order noise, but it also means the update magnitude must be tuned more carefully. At the baseline step size of 0.3, the batch method is simply too conservative.

## Learning-rate sweep and what it implies

| Batch variant | Final loss | Success rate (final loss $\leq 0.4$) | Mean cost to threshold |
|---|---|---|---|
| batch, lr=0.3 | $0.436 \pm 0.048$ | 0.20 | 98500 |
| batch, lr=1.0 | $0.365 \pm 0.036$ | 0.80 | 64788 |
| batch, lr=0.1 | $0.528 \pm 0.056$ | 0.00 | nan |
| batch, lr=0.05 | $0.592 \pm 0.043$ | 0.00 | nan |
| batch, lr=0.02 | $0.652 \pm 0.039$ | 0.00 | nan |
| batch, lr=0.3 inverse | $0.520 \pm 0.057$ | 0.00 | nan |
| batch, lr=1.0 inverse | $0.425 \pm 0.041$ | 0.20 | 50450 |

Table 2: Krotov batch hyperparameter sweep. Success rate is measured against the configured loss threshold.

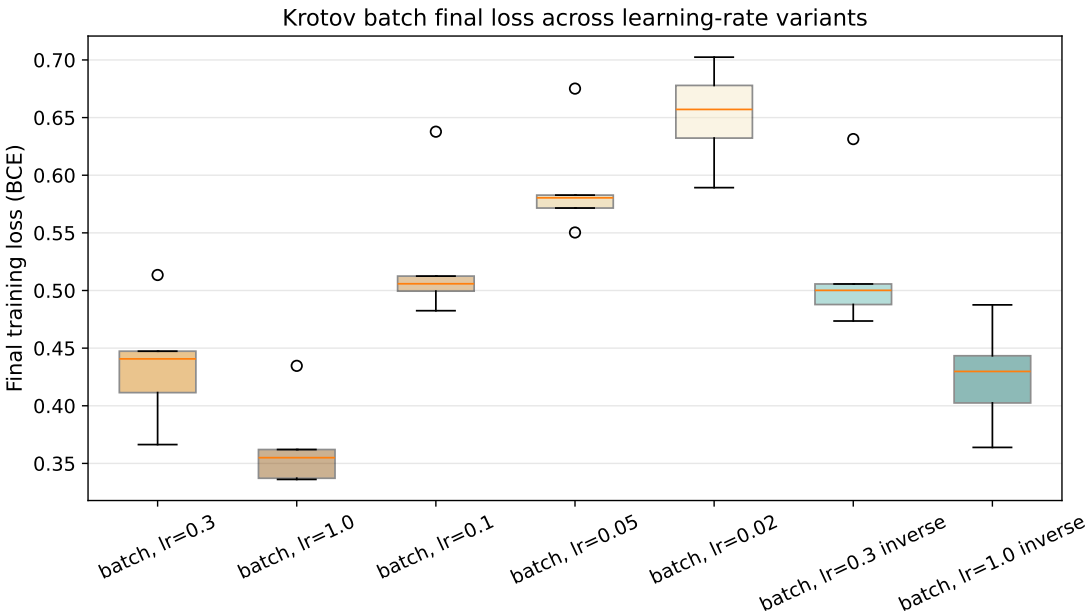


Figure 5: Distribution of final training losses across batch-step-size variants.

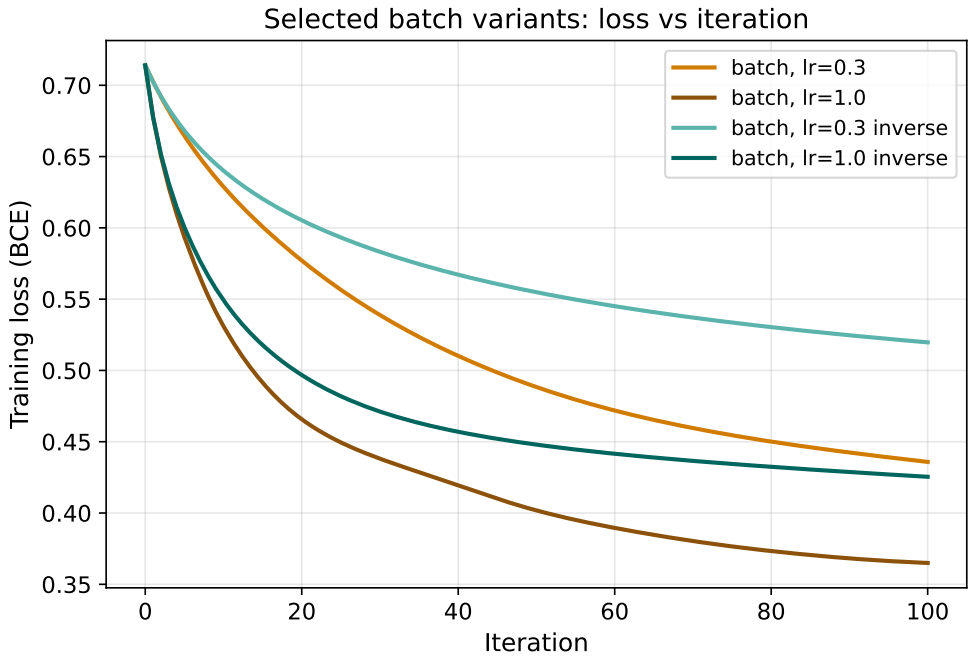


Figure 6: Selected batch variants. Larger initial step sizes recover much of the missing progress, while inverse decay trades speed for additional stability.

The sweep shows that the batch optimizer is not failing because of an obvious implementation bug. When the step size is increased, the batch method becomes significantly more competitive. In particular, the best-performing batch setting in this study is `batch, lr=1.0`, which reaches a mean final loss of 0.365. This is strong evidence that a large part of the apparent batch underperformance at the default setting came from step-size mismatch rather than from the BCE adjoint derivation or the fair-cost accounting.

At the same time, the sweep does *not* eliminate the qualitative difference between the online and batch dynamics. The online rule remains the more aggressive optimizer in the early phase, while the batch rule remains the smoother and more predictable one. The evidence therefore supports the following interpretation:

- unfair accounting was *not* the reason for the observed fast early drop;
- stale adjoints and single-sample noise are plausible explanations for the online oscillation;
- the baseline batch underperformance was strongly amplified by step-size choice;
- after retuning, the batch rule is viable, but its behavior is closer to stable first-order descent than to the highly expressive, sample-adaptive online Krotov trajectory.

## Selected batch variant versus Adam and L-BFGS-B

For the direct baseline check, the strongest tested batch setting in this sweep, `batch, lr=1.0`, is compared against the stored Adam and L-BFGS-B runs on the same five seeds used in the variant study (0, 1, 2, 3, 4). The older baseline JSON logs do not contain the newer fair-cost trace, so the overlay below uses the common saved axes shared by all three optimizers: iteration and wall-clock time.

| Optimizer | Final loss | Final test acc. | Wall time (s) | Total steps |
|---|---|---|---|---|
| batch, lr=1.0 | 0.365 ± 0.036 | 0.852 ± 0.015 | 52.9 ± 0.1 | 100.0 ± 0.0 |
| Adam | 0.353 ± 0.032 | 0.863 ± 0.025 | 97.8 ± 0.6 | 100.0 ± 0.0 |
| L-BFGS-B | 0.353 ± 0.029 | 0.837 ± 0.010 | 79.2 ± 21.1 | 78.8 ± 21.5 |

Table 3: Direct comparison on the aligned five-seed subset used in the Krotov-variant sweep.

The direct overlay shows that the retuned batch method is *close* to the gradient baselines, but not fully matched in final loss. On the aligned seeds, `batch, lr=1.0` reaches a mean final loss of 0.365 ± 0.036, versus 0.353 ± 0.032 for Adam and 0.353 ± 0.029 for L-BFGS-B. Its mean final test accuracy of 0.852 is similar to Adam's 0.863 and slightly above L-BFGS-B's 0.837. The strongest practical advantage of the batch Krotov variant is wall-clock speed: it finishes in 52.9s on average, compared with 97.8s for Adam and 79.2s for L-BFGS-B on this subset.

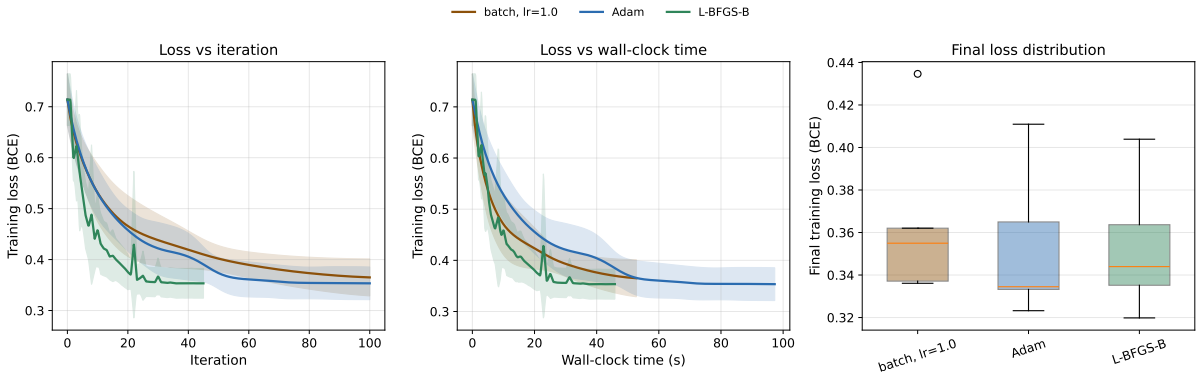


Figure 7: Selected batch Krotov variant (`lr=1.0`) against Adam and L-BFGS-B. The batch method is competitive in wall-clock time and test accuracy, while Adam and L-BFGS-B still retain a small edge in final training loss.

## Conclusion

The new Krotov variants separate two effects that were previously entangled. The preserved online optimizer confirms that the original method derives real benefit from sequential sample-wise updates, but it also exhibits the largest late-stage oscillations. The new full-batch variant removes the objective mismatch and the stale-adjoint sample ordering issue, yielding a cleaner optimization signal. However, it needs a substantially larger effective step size to compete on this benchmark. The overall picture is therefore not "implementation bug versus no bug", but rather "aggressive noisy online dynamics versus smoother objective-aligned batch dynamics, with learning-rate scale playing a decisive role."